

BEE 4750 Homework 1: Introduction to Using Julia

Name Natalie Ho

ID nh424

Due Date

Thursday, 9/11/25, 9:00pm

Overview

Instructions

- Problem 1 consist of a series of code snippets for you to interpret and debug. You will be asked to identify relevant error(s) and fix the code.
- Problem 2 asks you to write code to implement several simple functions using more general programming and some which are unique to Julia (which might mean that you need to look at the documentation or use Julia's help function, which is accessed with `?`, e.g. `?sum` for help with the `sum()` function) or to analyze Julia syntax.
- Problem 3 asks you to convert a verbal description of a wastewater treatment system into a Julia function, and then to use that function to explore the impact of different wastewater allocation strategies.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```

Activating project at `~/Downloads/BEE 4750/bee4750-hw1-nh424`
Precompiling project...
 711.7 ms  OpenSpecFun_jll
 747.7 ms  Rmath
1178.1 ms  ChainRulesCore
 989.9 ms  ArnoldiMethod
 644.5 ms  StaticArrays → StaticArraysStatisticsExt
 569.8 ms  Adapt → AdaptStaticArraysExt
 841.4 ms  QuadGK
3305.6 ms  ColorSchemes
 772.6 ms  ChainRulesCore → ChainRulesCoreSparseArraysExt
 636.2 ms  StaticArrays → StaticArraysChainRulesCoreExt
 913.7 ms  LogExpFunctions → LogExpFunctionsChainRulesCoreExt
2154.8 ms  SpecialFunctions
2045.3 ms  GeometryTypes
3910.2 ms  Graphs
2304.1 ms  Interpolations
 897.5 ms  HypergeometricFunctions
1515.8 ms  SpecialFunctions → SpecialFunctionsChainRulesCoreExt
 583.2 ms  ColorVectorSpace → SpecialFunctionsExt
9544.3 ms  GeometryBasics
1621.3 ms  StatsFuns
1522.7 ms  NetworkLayout
1472.2 ms  StatsFuns → StatsFunsChainRulesCoreExt
8421.3 ms  PlotUtils
1703.0 ms  NetworkLayout → NetworkLayoutGraphsExt
2375.6 ms  PlotThemes
4370.6 ms  Distributions
3395.2 ms  RecipesPipeline
16537.3 ms  Unitful
1101.2 ms  Distributions → DistributionsChainRulesCoreExt
1155.1 ms  Distributions → DistributionsTestExt
 725.9 ms  Unitful → PrintfExt
1093.2 ms  Interpolations → InterpolationsUnitfulExt
4060.2 ms  GraphRecipes
 810.8 ms  UnitfulLatexify
35211.5 ms  Plots
 3201.8 ms  Plots → GeometryBasicsExt
 3218.5 ms  Plots → UnitfulExt
 3368.2 ms  Plots → IJuliaExt
38 dependencies successfully precompiled in 58 seconds. 202 already precompiled.

```

Standard Julia practice is to load all needed packages at the top of a file. If you need to load

any additional packages in any assignments beyond those which are loaded by default, feel free to add a `using` statement, though [you may need to install the package](#).

```
using Random
using Plots
using GraphRecipes
using LaTeXStrings
using Distributions
```

```
# this sets a random seed, which ensures reproducibility of random number generation. You should
Random.seed!(1)
```

```
TaskLocalRNG()
```

Problems (Total: 30 Points)

Problem 1 (12 points)

The following subproblems all involve code snippets that require debugging. You are encouraged to use online resources (*e.g.* Julia documentation and forums, Stack Overflow, Reddit, etc) to help find diagnose error messages and find solutions, but make sure that you clearly document which resources you used and how you used them.

Problem 1.1

You've been tasked with writing code to identify the minimum value in an array. You cannot use a predefined function. Your colleague suggested the function below, but it does not return the minimum value.

```
function minimum(array)
    # initialize the minimum value counter
    min_value = array[1]
    # update minimum values
    for i in 1:length(array)
        if array[i] < min_value
            min_value = array[i]
        end
    end
    # return found minimum
    return min_value
end
```

```
array_values = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
@show minimum(array_values);
```

```
minimum(array_values) = 78
```

For this problem, we were tasked with writing code to identify the minimum value in an array. When I ran it the first time with the code provided, the output was reading that the min value was 0, when in reality the minvalue should be 78 based on the given array values. When looking through the code, the first line that jumped out to me was the `min_value = 0`. I assumed that this is where the code was wrong because it was immediately setting the minvalue to 0. Thus, I changed the code to have it read `min_value = array[1]` instead of 0. I did this because I knew I was trying to find the minvalue for a certain array of numbers so the code knew which minvalue it was trying to find. When I changed this line in the code, it produced the right answer of the minvalue of 78.

What is the logical flaw in the code? Fix it and show that your fixed code solves the problem.

Problem 1.2

Your team is trying to compute the average grade for your class, but the following code produces an `UndefVarError`.

```
# enter student grade vector
using Statistics
student_grades = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
# compute class average
function class_average(grades)
    return mean(grades)
end
average_grade = class_average(student_grades)
@show average_grade;
```

```
average_grade = 94.4
```

The `UndefVarError` in Julia means Undefined Variable Error. This happens when someone tries to use a variable that Julia does not recognize, meaning it has not been defined yet. This happens because `average_grade` only existed inside this function and not outside. To solve this problem, I added in a line above saying “using Statistics” so it knows the function of the global variable. From there, I was able to rework the ordering of the code and produce the correct output for finding the average student grades.

What does this `UndefVarError` mean? Why does this code produce one? Fix the error and solve the problem.

Problem 1.3

Your team wants to know the expected payout of an old Italian dice game called *passadieci* (which was analyzed by Galileo as one of the first examples of a rigorous study of probability). The goal of *passadieci* is to get at least an 11 from rolling three fair, six-sided dice. Your strategy is to compute the average wins from 1,000 trials, but the code you've written below produces a `MethodError`.

```
# function to simulate a passedieci roll: 3 6-sided dice
function passedieci()
    # this rand() call samples 3 values from the vector [1, 6]
    roll = rand(1:6, 3)
    return roll
end
# set number of trials and initialize outcome vector
n_trials = 1_000
outcomes = zeros(n_trials)
# simulate number of passedieci rolls and count wins
for i = 1:n_trials
    outcomes[i] = (sum(passadieci()) > 11)
end
win_prob = sum(outcomes) / n_trials # compute average number of wins
@show win_prob;
```

```
win_prob = 0.382
```

A method error in Julia occurs when you call a function with arguments that Julia does not know how to handle. The code produces this error in this line: `outcomes = zero(n_trials)`. The `zero(n_trials)` returns 0 meaning the outcomes is 0 and is a scalar not a vector. As it moves inside the loop, Julia has a problem because the `outcomes` thinks that it is going to be reading a vector but instead the outcome is just the number 0, so this is why we get a Method Error. To fix this problem we need a vector not a single integer so we can change `zero(n_trials)` instead to `zeros(n_trials)`. Note the difference is `zero` vs. `zeros`.

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

Problem 1.4

You're interested in writing some code to remove the mean of a vector from all of its components. You've written the following code and tried to test it on a random vector, but your code returns a `MethodError`.

```

using Statistics

# function to remove mean from a vector
function remove_mean(vect)
    function compute_mean(vect)
        element_sum = 0.0
        for v in vect
            element_sum += v
        end
        return element_sum / length(vect)
    end
    m = compute_mean(vect)
    return vect .- m
end

# test
random_vect = rand(1_000)
demeaned = remove_mean(random_vect)

@show round(mean(random_vect), digits=10)    # original mean
@show round(mean(demeaned), digits=10)      # demeaned mean (prints 0.0)

```

```

round(mean(random_vect), digits = 10) = 0.4854508546
round(mean(demeaned), digits = 10) = -0.0

```

```
-0.0
```

The method error I get when I run this code is “MethodError: no method matching (::Vector{Float64}, ::Float64)” this means that Julia looked for the definition of what is the problem but couldn’t find one. This is because in Julia this sign “-” between a vector and scalar is not directly defined. I have to remember that in Julia we need to have it format the subtraction sign as: `.-` for it to be defined as subtraction, similarly to what we have to do for multiplication. By changing the line of `-` to `.-` this fixes the problem so that we don’t have an error anymore.

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

Problem 2 (18 points)

Cheap Plastic Products, Inc. is operating a plant that produces $100\text{m}^3/\text{day}$ of wastewater that is discharged into Pristine Brook. The wastewater contains $1\text{kg}/\text{m}^3$ of YUK, a toxic

substance. The US Environmental Protection Agency has imposed an effluent standard on the plant prohibiting discharge of more than 20kg/day of YUK into Pristine Brook.

Cheap Plastic Products has analyzed two methods for reducing its discharges of YUK. Method 1 is land disposal, which costs $X_1^2/20$ dollars per day, where X_1 is the amount of wastewater disposed of on the land (m^3/day). With this method, 20% of the YUK applied to the land will eventually drain into the stream (*i.e.*, 80% of the YUK is removed by the soil).

Method 2 is a chemical treatment procedure which costs \$1.50 per m^3 of wastewater treated. The chemical treatment has an efficiency of $e = 1 - 0.005X_2$, where X_2 is the quantity of wastewater (m^3/day) treated. For example, if $X_2 = 50\text{m}^3/\text{day}$, then $e = 1 - 0.005(50) = 0.75$, so that 75% of the YUK is removed.

Cheap Plastic Products is wondering how to allocate their wastewater between these three disposal and treatment methods (land disposal, chemical treatment, and direct disposal) to meet the effluent standard while keeping costs manageable.

The flow of wastewater through this treatment system is shown in Figure 1. Modify the edge labels (by editing the `edge_labels` dictionary in the code producing Figure 1) to show how the wastewater allocations result in the final YUK discharge into Pristine Brook. For the `edge_label` dictionary, the tuple (i, j) corresponds to the arrow going from node i to node j . The syntax for any entry is $(i, j) \Rightarrow \text{"label text"}$, and the label text can include mathematical notation if the string is prefaced with an L, as in `L"x_1"` will produce x_1 .

```
using Plots
using GraphRecipes
using LaTeXStrings

A = [0 1 1 1;
      0 0 0 1;
      0 0 0 1;
      0 0 0 0]

names = ["Plant", "Land Treatment", "Chem Treatment", "Pristine Brook"]

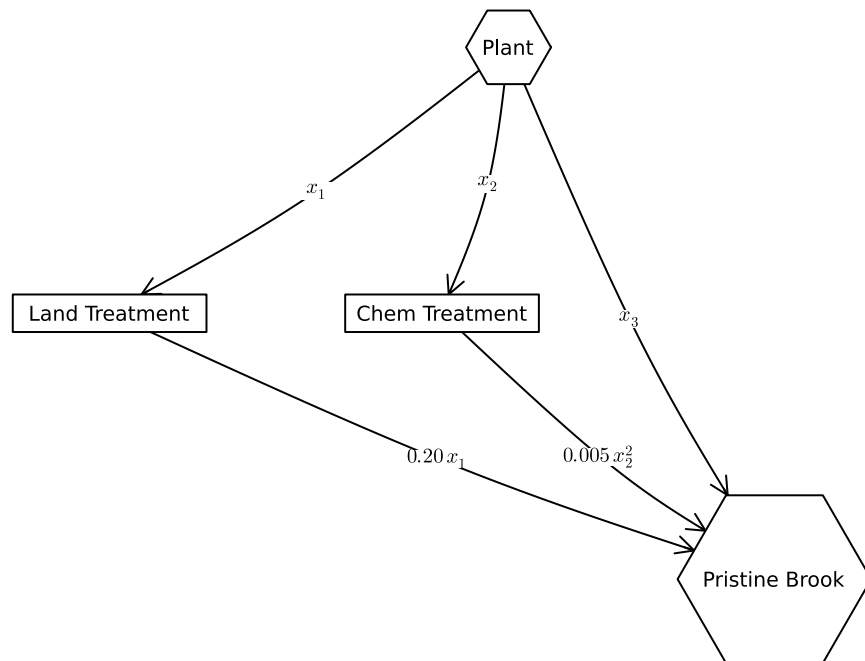
# Edge labels (use LaTeXStrings everywhere)
edge_labels = Dict{
    (1, 2) => L"x_1",           # Plant → Land
    (1, 3) => L"x_2",           # Plant → Chem
    (1, 4) => L"x_3",           # Plant → Brook
    (2, 4) => L"0.20\",x_1",     # Land → Brook (YUK contribution)
    (3, 4) => L"0.005\",x_2^{2}" # Chem → Brook (YUK contribution)
}
```

```

shapes = [:hexagon, :rect, :rect, :hexagon]
xpos   = [0, -1.5, -0.25, 1.0]
ypos   = [1.0, 0.0, 0.0, -1.0]

p = graphplot(
    A; names=names, edgelabel=edge_labels, markersize=0.15, markershapes=shapes, markercolor=
)
display(p)

```



Problem 2.1

Formulate a mathematical model for the treatment cost and the amount of YUK that will be discharged into Pristine Brook based on the wastewater allocations. This is best done with some equations and supporting text explaining the derivation. Make sure you include, as additional equations in the model, any needed constraints on relevant values. You can find some basics on writing mathematical equations using the LaTeX typesetting syntax [here](#), and a cheatsheet with LaTeX commands can be found on the course website's [Resources page](#).

Problem 2.2

Implement your systems model as a Julia function which computes the resulting YUK discharge and cost for a particular treatment plan. You can return multiple values from a function with a [tuple](#), as in:

Problem 2.1- Mathematical Model

Variables (m³/day)

Decision variables (m³/day):

- x_1 : flow to land disposal
- x_2 : flow to chemical treatment
- x_3 : flow to direct discharge

Parameters

- $Q = 100\text{m}^3/\text{day}$
- $C_0 = 1\text{kg}/\text{m}^3$
- $Y = 20\text{kg}/\text{day}$

Mass Balance

$$x_1 + x_2 + x_3 = Q, \quad x_i \geq 0 \ (i = 1, 2, 3).$$

Cost Model (USD/day)

Land: $X_1^2/20$; Chemical: \$1.50 per m³; Direct: 0.

$$C(x_1, x_2) = \frac{x_1^2}{20} + 1.5 x_2.$$

YUK Discharged (kg/day)

Land leaves 20% of YUK:

$$Y_{\text{land}} = 0.20 C_0 x_1.$$

Chemical leaves fraction (1-e = 0.005,x_2):

$$Y_{\text{chem}} = (1 - e) C_0 x_2 = 0.005 C_0 x_2^2.$$

Direct discharge:

$$Y_{\text{direct}} = C_0 x_3.$$

Total discharge:

$$Y(x_1, x_2, x_3) = 0.20 x_1 + 0.005 x_2^2 + x_3.$$

Regulatory Constraint

$$Y(x_1, x_2, x_3) \leq Y_{\text{max}} = 20.$$

Summary of the Mathematical Models

Total Cost

$$C(x_1, x_2) = \frac{x_1^2}{20} + 1.5 x_2$$

YUK Discharge

$$\begin{aligned} x_1 + x_2 + x_3 &= 100, \\ 0.20 x_1 + 0.005 x_2^2 + x_3 &= Y, \\ x_1, x_2, x_3 &\geq 0. \end{aligned}$$

I created this mathematical model by first translating the wastewater treatment problem into descriptive relationships that represent how the system behaves. The decision variables represent the flows sent to land disposal, chemical treatment, and direct discharge, and a mass balance ensures that the entire daily wastewater flow is accounted for. The cost relationship reflects that land disposal becomes increasingly expensive as more flow is sent there, showing rising marginal costs, while chemical treatment maintains a constant cost per unit and direct discharge has no cost. The discharge relationship describes how each treatment method affects pollution removal differently—land disposal removes most of the pollutant but leaves some behind, chemical treatment becomes more effective as larger volumes are treated, and direct discharge leaves everything untreated. A regulatory limit ties this discharge relationship to the environmental standard for maximum allowable pollutant release. For formatting, I referenced the starter code for Problem 2 to organize my Markdown sections and then asked ChatGPT to refine the equation formatting so the explanation would be clear and professional, specifically

using LaTeX as I have never used it before, and I am looking to learn. Together, these descriptions define a complete mathematical model of the problem as they capture the flows, costs, and pollutant discharge while establishing the feasible solutions that meet the regulatory requirement.

```
# Problem 2.2

const Q = 100.0 # m^3/day
const c0 = 1.0 # kg/m^3
const Y_LIMIT = 20.0

"""
    yuk_variables(x1, x2, x3)

- Y = YUK discharged to Pristine Brook (kg/day)
- C = total treatment cost (\$/day)

flow/mass balance equation: x1 + x2 + x3 == Q.
"""
function multiple_return_variables(x1::Real, x2::Real, x3::Real)

    # Cost
    C = (x1^2)/20 + 1.5*x2

    # YUK discharge
    Y = 0.20*x1 + 0.005*x2^2 + x3

    return (Y, C)
end

# Example to test code
Y, C = multiple_return_variables(40.0, 30.0, 30.0) # x1 + x2 + x3 = 100
@show Y
@show C
```

```
Y = 42.5
C = 125.0
```

```
125.0
```

To evaluate the function over vectors of inputs, you can *broadcast* the function by adding a decimal `.` before the function arguments and accessing the resulting values by writing a *comprehension* to loop over the individual outputs in the vector:

When I first ran the code, it tried to call `multiple_return_values` on the vectors `x` and `y`, but I hadn't actually defined that function yet in the notebook. Because of that, Julia threw an `UndefVarError`, implying it didn't know what function to use. The rest of the code looping through each tuple with list comprehensions worked, but none of it could run without the function being set up first.

In the fixed version, I added the definition for `multiple_return_values(x, y)` so it returns a tuple with the sum and product of its two inputs. Using broadcasting with `multiple_return_values.(x, y)` then applies it element by element across the two vectors. Inside the function, `x` and `y` are just scalars, so `x + y` and `x * y` work as intended. The broadcast gives back a vector of tuples, and the list comprehensions pick out the first values as `a` and the second values as `b`. With my vectors, that gives `a = [7, 9, 11, 13, 15]` and `b = [6, 14, 24, 36, 50]`, which shows the corrected code now works the way I wanted.

```
function multiple_return_values(x,y)
    return (x+y, x*y)
end
x = [1, 2, 3, 4, 5]
y = [6, 7, 8, 9, 10]

output = multiple_return_values.(x, y)
a = [out[1] for out in output]
b = [out[2] for out in output]
@show a;
@show b;
```

```
a = [7, 9, 11, 13, 15]
b = [6, 14, 24, 36, 50]
```

Make sure you comment your code appropriately to make it clear what is going on and why.

Problem 2.3

Use your function to experiment with 1,000 different combinations of wastewater discharge and treatment. You can do this with either a grid search or by sampling from a [Dirichlet distribution](#) (a $\text{Dirichlet}(1, n)$ distribution will generate uniformly-weighted n -dimensional vectors whose components add up to 1; see the [Distributions.jl documentation](#) for how to sample from probability distributions in Julia). Plot the results of these experiments. Do any satisfy the YUK effluent standard (plot this as well as a dashed red line). What was the cost of solutions satisfying the standard? What can you say about the tradeoff between treatment cost and YUK discharge? You don't have to find an "optimal" solution to this problem, but what do you think would be needed to find a better solution?

Problem 2.4

Find the strategies which minimize cost and YUK discharge (these will be different strategies) analytically and find the values of the objective metrics. Plot these values in the plot that you created for Problem 2.3. How do their values compare to the spread of values that you found in that problem? Would you select either of them (explain why or why not)?

References

List any external references consulted, including classmates.

```
using Distributions
using Random
using Plots

const Q = 100.0
const Y_LIMIT = 20.0

function multiple_return_values(x1::Real, x2::Real, x3::Real)
    C = (x1^2)/20 + 1.5*x2
    Y = 0.20*x1 + 0.005*x2^2 + x3
    return (Y, C)
end

Random.seed!(123)

n = 1000
d = Dirichlet(fill(1.0, 3))
W = rand(d, n)

x1 = Q .* vec(W[1, :])
x2 = Q .* vec(W[2, :])
x3 = Q .* vec(W[3, :])

Ys = similar(x1)
Cs = similar(x1)
for i in eachindex(x1)
    Ys[i], Cs[i] = multiple_return_values(x1[i], x2[i], x3[i])
end

feas = Ys .<= Y_LIMIT
@show count(feas)
```

```

if any(feas)
    @show minimum(Cs[feas])
end

# --- Plot Y (x-axis) vs C (y-axis) ---
p = scatter(Ys, Cs; ms=3, alpha=0.5,
            xlabel="YUK discharged (kg/day)",
            ylabel="Cost (USD/day)",
            label="Random plans")

# effluent limit is vertical at Y = Y_LIMIT
vline!(p, [Y_LIMIT]; l=:dash, label="Effluent limit (Y = $(Y_LIMIT))")

# highlight cheapest feasible point (if any)
if any(feas)
    feas_idx = findall(feas)
    _, local_min = findmin(Cs[feas])
    best_idx = feas_idx[local_min]
    bestC, bestY = Cs[best_idx], Ys[best_idx]
    @show bestC bestY

    scatter!(p, [bestY], [bestC];
            markershape=:star5, ms=12, color=:red,
            label="Cheapest feasible")
end

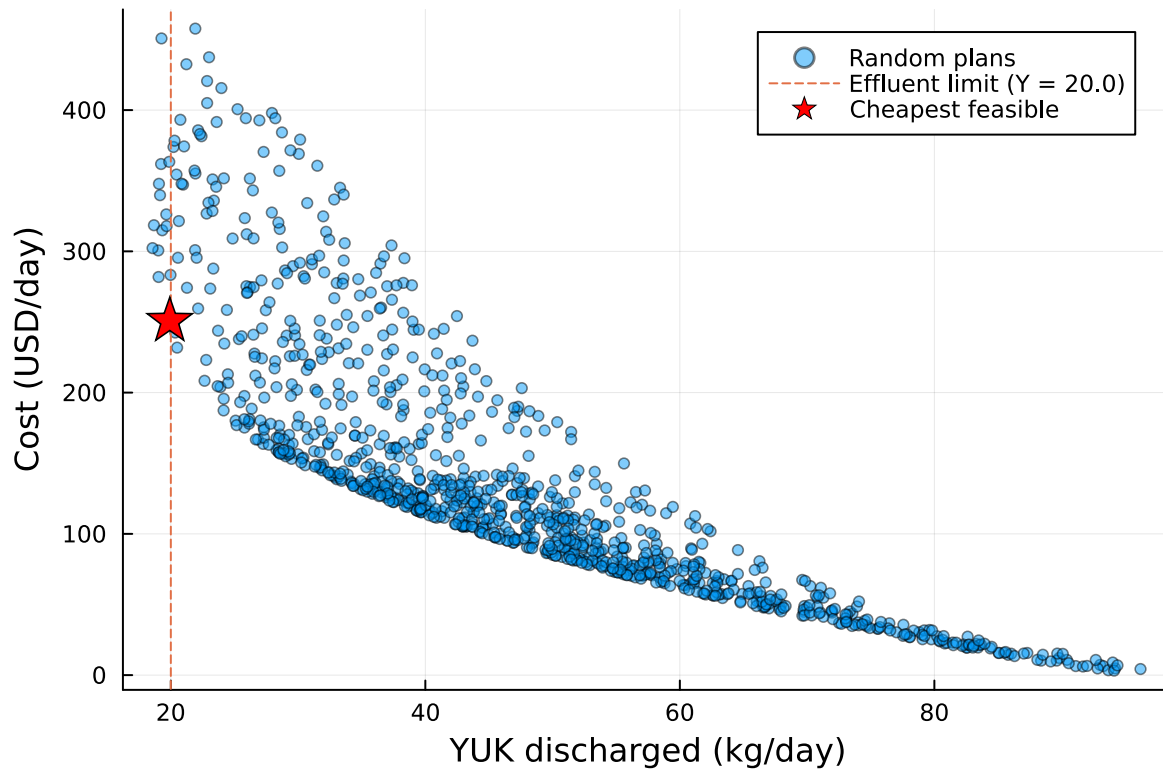
display(p)

```

```

count(feas) = 14
minimum(Cs[feas]) = 250.6827077579617
bestC = 250.6827077579617
bestY = 19.91666863776414

```



Commenting the Code Above

1. Import packages

Loads `Distributions` (for random splits of flow), `Random` (to control randomness), and `Plots` (to make the figure).

2. Set constants

Defines the total daily flow $Q = 100.0$ (m^3/day) and the discharge cap $Y_LIMIT = 20.0$ (kg/day).

3. Define the model

`multiple_return_values(x1, x2, x3)` computes, for one plan:

- **Cost** $C = x1^2/20 + 1.5*x2$
- **YUK** $Y = 0.20*x1 + 0.005*x2^2 + x3$
(Land has quadratic cost and leaves 20% of pollutant; chemical is \$1.5 per m^3 with a quadratic residual; direct discharge is untreated and free.)

4. Make results reproducible

`Random.seed!(123)` fixes the random number generator so you get the same samples each time.

5. Sample random allocation plans

Set `n = 1000`. Draw from a Dirichlet distribution with 3 components: `Dirichlet(fill(1.0, 3))`.

Each draw is a triple of nonnegative fractions that sum to 1 (a randomized split across land/chemical/direct).

6. Scale to physical flows

Convert fractions to m^3/day by multiplying by `Q`, producing vectors:

- `x1` (to land), `x2` (to chemical), `x3` (to direct).
- By construction, every plan satisfies $x1 + x2 + x3 = Q$.

7. Evaluate each plan

For every index `i`, call the model and store:

- `Ys[i]` = YUK discharge for plan `i`
- `Cs[i]` = cost for plan `i`

8. Flag feasible plans

Create `feas = Ys .<= Y_LIMIT` to mark plans that meet the cap.

Print the number of feasible plans, and if any exist, print the minimum cost among them.

9. Plot all plans

Make a scatter plot with **Y** on the x-axis (kg/day) and **Cost** on the y-axis (USD/day). Points are semi-transparent to show density.

10. Show the effluent limit

Draw a dashed **vertical** line at `Y = 20` to indicate the regulatory cap.

11. Highlight the cheapest feasible plan

If feasible plans exist, find the index of the smallest `Cs[feas]`, map it back to the full arrays, and overlay a red star at that plan's (`Y`, `C`).

12. Display the figure

`display(p)` renders the finished plot.

These results indicate that only **14 out of 1000** sampled plans met the discharge cap (`Y <= 20`), which shows that feasible allocations are rare when direct discharge is allowed in the sampling. The **cheapest feasible sample** landed at (**Y 19.9167**) **kg/day** (just under the cap) with a **cost (\$250.68)/day**. This is about **\$10.68/day** higher than the analytic minimum of **\$240.00/day** achieved at the mix $((x_1, x_2, x_3)=(60, 40, 0))$, where direct discharge is eliminated. The gap is expected: random three-way splits rarely land exactly on the optimum, and any positive (x_3) forces more treatment in (x_1) or (x_2) to stay under the cap—raising cost. To surface the true optimum more reliably, either sample only two routes with $(x_3=0)$, increase the number of samples, or plot the analytic point $((Y, C)=(20, 240))$ directly.

To minimize cost, there are two possible interpretations depending on whether the environmental discharge limit is enforced. If the YUK limit is ignored completely, the absolute cheapest option would be to send all 100 m³/day of wastewater directly to Pristine Brook without any treatment. This choice has zero cost but is clearly unacceptable because it violates the discharge standard. When the 20 kg/day YUK cap is enforced, the strategy changes. Direct discharge releases the entire pollutant load untreated, so using it would quickly exceed the cap and force more expensive treatment elsewhere, ultimately raising costs. To avoid that, direct discharge is eliminated by setting $x_3=0$. The remaining flow is split between land disposal and chemical treatment, with the discharge constraint made exactly active at 20 kg/day. If discharge were reduced below this limit, the cost could be lowered by treating less, so the minimum-cost solution must be on the boundary. Solving the mass balance and discharge equations gives an allocation of 60 m³/day to land disposal and 40 m³/day to chemical treatment, which results in a total cost of 240 USD/day. This combination is the lowest-cost feasible strategy under the YUK discharge requirement.

To get the lowest possible YUK discharge, I would ignore cost and focus only on keeping pollution out of Pristine Brook. The first step is to completely avoid direct discharge since that would send the full pollutant load straight to the brook with no treatment. That leaves just land disposal and chemical treatment. Land disposal always leaves a small, fixed amount of YUK behind, while chemical treatment does a good job at smaller flows but starts to perform worse if too much water is pushed through it. Balancing those effects, the cleanest split is about 80 m³/day to land disposal and 20 m³/day to chemical treatment. That combination brings the total discharge down to around 18 kg/day, which is below the 20 kg/day limit and represents the best we can do with this setup. It's essentially the point where both methods are working in their most effective range without overloading either option.

Problem 2.4 - Minimizing the Mathematical Model

Variables (m³/day)

Decision variables (m³/day):

- x_1 : flow to land disposal
- x_2 : flow to chemical treatment
- x_3 : flow to direct discharge

Parameters

- $Q = 100\text{m}^3/\text{day}$

- $C_0 = 1\text{kg}/\text{m}^3$
- $Y = 20\text{kg}/\text{day}$

Mass Balance

$$x_1 + x_2 = Q, \quad x_3 = 0, \quad x_1, x_2 \geq 0.$$

Cost Model (USD/day)

Land: $X_1^2/20$; Chemical: \$1.50 per m^3 ; Direct: 0.

$$C(x_1, x_2) = \frac{x_1^2}{20} + 1.5 x_2.$$

YUK Discharged (kg/day)

Land leaves 20% of YUK; chemical residual is quadratic in treated flow.

$$Y(x_2) = 0.20 (Q - x_2) + 0.005 x_2^2.$$

Regulatory Constraint

$$Y(x_2) \leq Y_{\max} = 20.$$

Summary of the Mathematical Models

Total New Cost Minimized

$$C(x_2) = \frac{(Q - x_2)^2}{20} + 1.5 x_2$$

YUK Discharge Minimized

$$x_1 + x_2 = Q, \quad x_3 = 0, \quad Y(x_2) = 0.20 (Q - x_2) + 0.005 x_2^2, \quad x_1, x_2 \geq 0.$$

```

using Distributions
using Random
using Plots

const Q = 100.0
const Y_LIMIT = 20.0

# New equations (no x3 term in Y)
function multiple_return_values(x1::Real, x2::Real, x3::Real)
    C = (x1^2)/20 + 1.5*x2
    Y = 0.20*x1 + 0.005*x2^2 # x3 eliminated from Y
    return (Y, C)
end

Random.seed!(123)

# --- replace 3-way Dirichlet with 2-way (land & chemical only) ---
n = 1000
d2 = Dirichlet(fill(1.0, 2))
W2 = rand(d2, n)

x1 = Q .* vec(W2[1, :]) # land
x2 = Q .* vec(W2[2, :]) # chemical
x3 = zeros(length(x1)) # direct discharge = 0

# Evaluate Y and C for all samples
out = multiple_return_values.(x1, x2, x3) # vector of tuples
Ys = getindex.(out, 1)
Cs = getindex.(out, 2)

# Feasibility under the cap
feas = Ys .<= Y_LIMIT
@show count(feas)
if any(feas)
    @show minimum(Cs[feas])
end

# --- Plot Y (x-axis) vs C (y-axis) ---
p = scatter(Ys, Cs; ms=3, alpha=0.5,
            xlabel="YUK discharged (kg/day)",
            ylabel="Cost (USD/day)",
            label="Random plans (x=0)")

```

```

# Effluent limit is vertical at Y = Y_LIMIT
vline!(p, [Y_LIMIT]; l=:dash, label="Effluent limit (Y = $(Y_LIMIT))")

# Highlight cheapest feasible point (if any)
if any(feas)
    feas_idx = findall(feas)
    _, local_min = findmin(Cs[feas])
    best_idx = feas_idx[local_min]
    bestC, bestY = Cs[best_idx], Ys[best_idx]
    @show bestC bestY

    scatter!(p, [bestY], [bestC];
             markershape=:star5, ms=12, color=:red,
             label="Cheapest feasible")
end

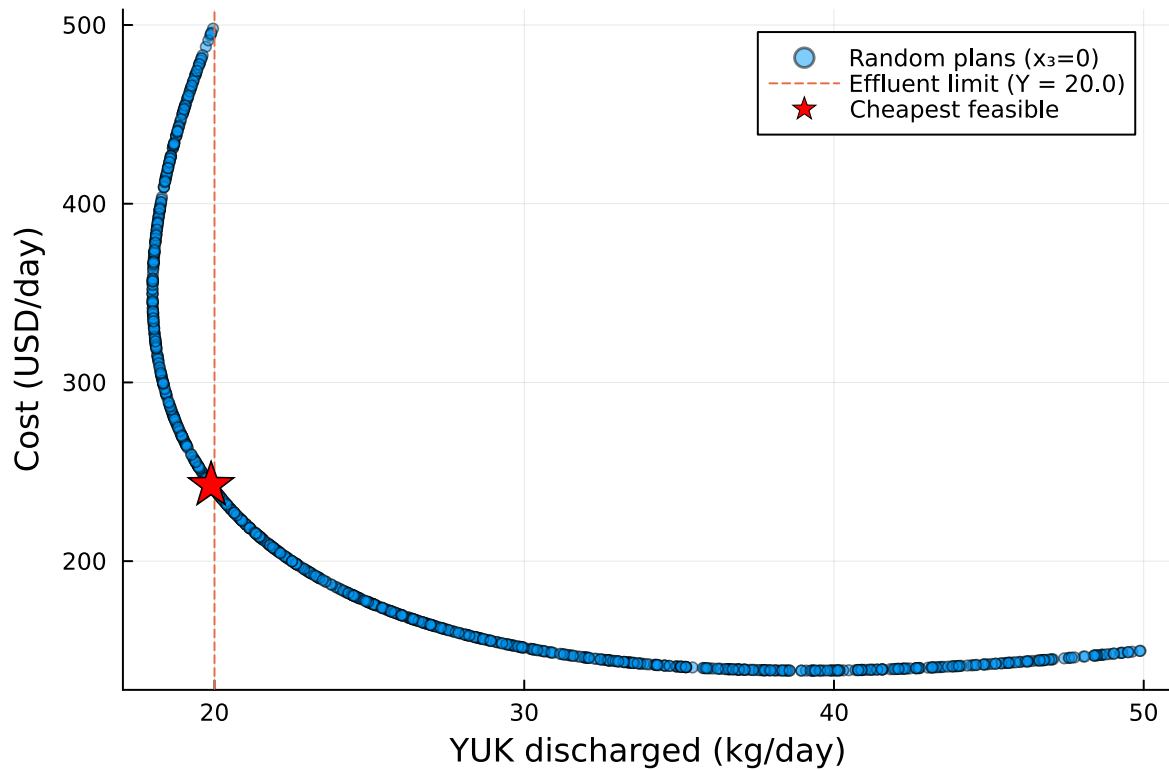
display(p)

```

```

count(feas) = 370
minimum(Cs[feas]) = 242.50700896989827
bestC = 242.50700896989827
bestY = 19.89079164694922

```



When I look at my scatter, the random plans form a big cloud and the two “best” solutions sit along the bottom edge (lowest cost for a given Y). The cost-minimizer is at $Y = 20$ kg/day with cost USD 240/day—it’s the cheapest plan that still meets the rule. My best sampled feasible plan is $Y = 19.92$ kg/day with cost USD 250.68/day—slightly cleaner but about USD 10–11 more per day. The Y -minimizer is $Y = 18$ kg/day with cost USD 350/day—cleaner, but roughly USD 110/day more than the cost-minimizer. If the goal is simply to meet the cap as cheaply as possible, I’d pick the cost-minimizer; in practice, I’d prefer a tiny safety buffer and choose something near $Y = 19.9$ for a modest premium. I wouldn’t choose the Y -minimizer unless that extra ~ 2 kg/day of headroom is truly necessary, because the cost jump is large for a small gain.